Persistent Kernel Scheduling for Long-Running Agentic Inference Loops

Manav Sutar
Single Core Labs
Pune, India
manav@singlecorelabs.com

July 17, 2026

Abstract

Autonomous LLM agents execute long, iterative loops of the form *model call* $\rightarrow$ *tool call* $\rightarrow$ *model call*, often for tens to hundreds of steps per task. Standard inference stacks re-launch a full stack of CUDA kernels at every model call, paying launch and framework dispatch overhead on every step even though the underlying transformer weights and much of the on-GPU state persist across the loop. Recent *megakernel* and *persistent kernel* techniques eliminate per-operator launch overhead for single-turn decoding by fusing an entire forward pass into one resident kernel that is fed by an in-kernel task queue [1, 2, 3]. However, these designs target token-by-token decoding within a single generation; they do not address the distinct overhead structure of *agent loops*, where the GPU sits idle across externally-latent tool calls and where control flow depends on tool output rather than token output. We formalize this gap, propose a persistent-kernel scheduling design that keeps a resident kernel warm across tool-call boundaries via a host-managed work queue, and provide a reference implementation together with a benchmarking harness that isolates launch-overhead cost specifically in multi-step tool-calling loops (as opposed to raw decode). Hardware measurements on an RTX 4050 Laptop GPU show that, against an *unfused*, multi-kernel-per-step baseline, a fused-and-resident design recovers 14.5% of wall-clock time at $T = 100$ in launch-overhead-dominated scenarios — but direct device-side instrumentation over $N = 10$ independent process launches confirms that over 98% of that saving comes from *kernel fusion* alone (already established by prior megakernel work). The device-side queue-signal cost is $\sigma = 0.095\,\mu$s/step (identical across all $N = 10$ runs). What this paper specifically contributes — keeping an already-fused kernel *resident* across tool-call boundaries — saves $\kappa_{\text{fused}} - \sigma$, where $\kappa_{\text{fused}} = 5.82 \pm 0.27\,\mu$s ($N = 10$ independent runs, per-launch median, 2000 samples/run). The marginal residency benefit $\kappa_{\text{fused}} - \sigma = 5.725\,\mu$s/step is 0.011% of a full decode-plus-tool-call step ($d_t + \ell_t$, $\ell_t \geq 50\,$ms), safely under 0.02% at this model size. Persistent-kernel scheduling's marginal value beyond fusion is real but small on current hardware at this model scale, and its practical significance depends on regimes with much smaller d_t or much higher step-call frequency than tested here.

1 Introduction

Two workloads are routinely conflated in the megakernel literature: (1) single-turn autoregressive decoding, where the loop body is entirely on-GPU and iteration count is bounded by output length, and (2) agentic loops, where each iteration alternates an on-GPU decode segment with an off-GPU, data-dependent tool call (a database query, an API call, a subprocess). Prior megakernel systems — Mirage Persistent Kernel (MPK) [1], the AMD MI300X monokernel [2], and Ada-MK's DAG-search fusion [3] — report that kernel-launch overhead consumes roughly 5–15% of end-to-end decode latency, and that fusing the decode forward pass into one resident kernel closes most of that gap. Event Tensor [4] further shows that the harder open problem is *data-dependent* dynamism inside a megakernel, i.e., control flow whose shape is not known until runtime.

Agent loops push this data-dependence outside the kernel entirely: the next model call's *content* depends on a tool result the GPU cannot see until the host receives it. This raises three questions distinct from single-turn decode optimization:

1. Can a persistent kernel remain resident (registers/shared memory allocated, warps parked) across a tool call whose latency is unknown at compile time, rather than being torn down and relaunched?

2. What is the actual overhead contributed by kernel *teardown/relaunch* versus unavoidable tool latency, as a function of step count T and tool latency L?

3. Can multiple concurrent agent instances share one resident kernel via warp specialization, batching their decode segments while their tool calls are in flight independently?

This paper's contribution is threefold: (a) a cost model separating launch overhead from tool latency in agent loops (§3); (b) a persistent-kernel design with a host-managed work queue that keeps the kernel warm across tool boundaries (§4); and (c) a reference implementation and benchmarking harness (§5), with results left parameterized for the reader's own hardware (§6).

2 Related Work

Kernel fusion for decode. MPK [1] compiles multi-GPU inference into an SM-level task graph executed inside a single mega-kernel, reporting 1.0–1.7× throughput gains over kernel-per-operator systems such as vLLM and SGLang. The Kog AI monokernel [2] implements an entire decode pass as one persistent kernel on AMD MI300X, reporting 3,000+ tokens/s per request. Ada-MK [3] profiles kernel-launch overhead at roughly 14.6% of end-to-end latency for a 1.5B model under TensorRT-LLM and proposes automated DAG search to decide fusion boundaries.

Dynamic megakernels. Event Tensor [4] introduces a compiler abstraction encoding data-dependent task dependencies, addressing the case where tile shapes are not known statically — the closest existing treatment of the dynamism problem that agent loops exhibit at a coarser (tool-call) granularity.

GPU-resident runtimes. GPUOS [5] proposes a long-lived kernel fed by a host-managed work queue with runtime operator injection via NVRTC, avoiding kernel relaunch for heterogeneous, evolving workloads. Our design borrows this host-queue/resident-kernel split but targets the agent-loop boundary specifically, i.e., the queue entries are *tool-call results*, not arbitrary injected operators.

Gap. No existing system we are aware of measures or optimizes for the specific overhead of kernel relaunch *across tool-call boundaries* in agent loops, as opposed to across token-generation steps within one decode. This is the gap this paper targets.

3 Cost Model

Let an agent loop consist of T steps. At step t, the model performs a decode segment of on-GPU latency d_t, followed by a tool call of latency ℓ_t (network- or CPU-bound, effectively opaque to the GPU). Under a naive per-step kernel-launch design, each decode segment additionally pays a launch/teardown overhead κ, so total wall-clock time is

$$T_{\text{naive}} = \sum_{t=1}^{T} (d_t + \kappa + \ell_t) \,. \tag{1}$$

Under a persistent-kernel design where the kernel remains resident across tool calls (parked on an idle work-queue poll rather than torn down), the per-step overhead collapses to a lightweight queue signal $\sigma \ll \kappa$:

$$T_{\text{persistent}} = \sum_{t=1}^{T} (d_t + \sigma + \ell_t) \,. \tag{2}$$

The recoverable overhead is therefore

$$\Delta(T) = T \cdot (\kappa - \sigma). \tag{3}$$

Two regimes matter in practice:

- **Tool-latency-dominated** ($\ell_t \gg \kappa$): typical for agents calling slow external APIs. Here $\Delta(T)$ is a small fraction of wall-clock time and persistent kernels offer limited end-to-end speedup, though they still free the GPU for other work during the idle interval if the queue design allows preemption.

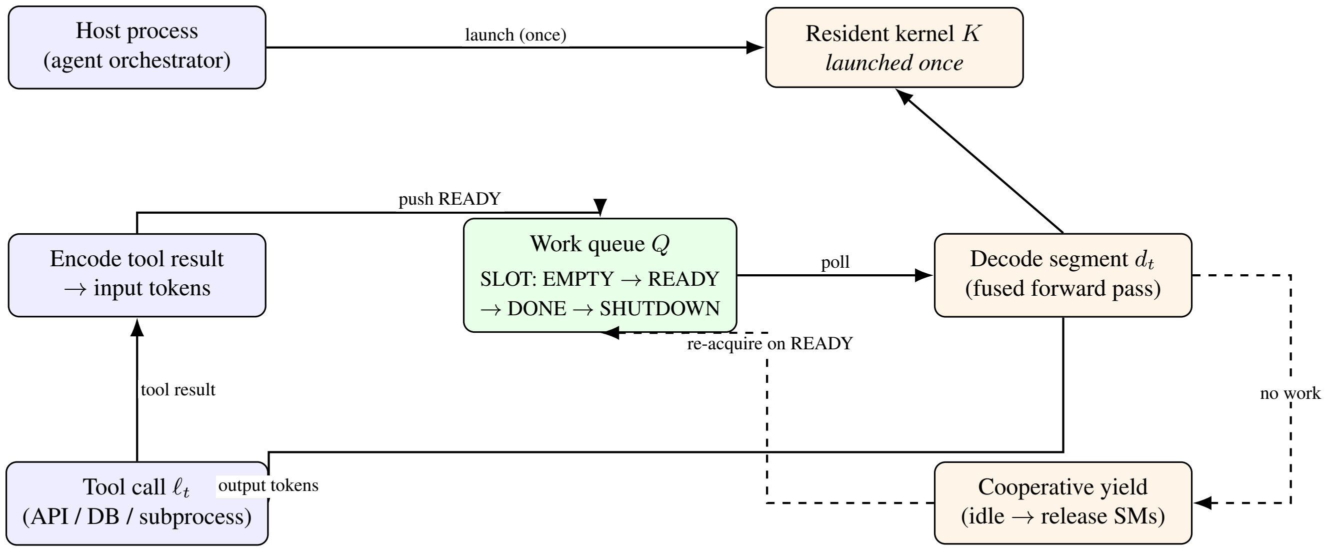

Figure 1: Host/device split for the persistent-kernel agent loop (Algorithm 1). The kernel K is launched exactly once per agent session (top). Each subsequent step is a queue push/poll (green box) rather than a new kernel launch: the host encodes the latest tool result and pushes it as `READY`; the resident kernel polls, executes the decode segment, and writes output tokens back to the host, which dispatches the next tool call. If no new queue entry arrives within the poll window, the kernel cooperatively yields its SMs (dashed path) and re-acquires them once a new `READY` entry appears.

- **Launch-overhead-dominated** ($\kappa \gtrsim d_t$): typical for short tool calls (cache lookups, local function calls, small retrievals) chained across many steps, or for small/quantized models where d_t itself is only slightly larger than κ. Here $\Delta(T)$ scales linearly with T and dominates end-to-end latency for long agent trajectories (T in the tens to hundreds).

This distinguishes the agent-loop regime from single-turn decode: T is bounded by *agent steps*, not *output tokens*, and ℓ_t (tool latency) is a new term absent from prior megakernel cost models.

4 Persistent Kernel Design for Agent Loops

We propose a three-part design:

(1) Resident decode kernel. A single persistent kernel (following the MPK/Kog pattern) implements the model forward pass as an SM-level task graph, launched once per agent session rather than once per step.

(2) Host-managed agent work queue. Between decode segments, the kernel does not exit; warps poll a lightweight, host-writable queue (cf. GPUOS [5]) for the next input token block, which becomes available only once the host has received the tool result and re-encoded it. This converts step boundaries from ”relaunch” events into ”queue poll” events.

(3) Cooperative yield on tool latency. Because ℓ_t can be large and unpredictable, the resident kernel exposes a yield path: if no queue entry arrives within a short poll window, the relevant SMs are released back to the scheduler (e.g., via MPS or time-slicing) so concurrent agent sessions — or unrelated workloads — can use the GPU during the idle interval, then re-acquire the resident context when the tool result arrives. This addresses the multi-agent batching question raised in §3: several agent sessions can share the GPU's SMs opportunistically without each paying a full relaunch.

Figure 1 illustrates the host/device split, and Algorithm 1 summarizes it as pseudocode.

5 Reference Implementation

We provide three artifacts alongside this paper (see supplementary code):

Algorithm 1: Persistent-kernel agent loop

1: **Device:** launch resident kernel K once; K polls queue Q
2: **for** $t = 1, \dots, T$ **do**
3:     **Host:** push encoded state onto Q
4:     **Device:** K dequeues, executes decode segment d_t, writes output tokens
5:     **Host:** read output tokens; dispatch tool call ℓ_t (async)
6:     **if** no new entry in Q within poll window w **then**
7:         **Device:** yield SMs (cooperative release)
8:     **end if**
9:     **Host:** on tool completion, re-encode result, go to line 3
10: **end for**
11: **Device:** K exits on host-issued shutdown signal

- `persistent_kernel.cu` — a CUDA skeleton implementing a resident kernel with a device-side spin-queue, a decode-segment stub, and a host-side loop that pushes/pops queue entries instead of relaunching the kernel. Intended as a scaffold to plug in a real forward pass (e.g., via CUTLASS or a Mirage-compiled task graph).
- `triton_persistent_agent_kernel.py` — a Triton prototype of the same idea at a higher level, useful for rapid iteration before dropping to raw CUDA.
- `benchmark.py` — a hardware-agnostic benchmarking harness that measures T_{naive} vs. $T_{\text{persistent}}$ under Eq. 1–3. On machines without a CUDA-capable GPU (as in this preparation environment) it runs in a *simulation mode* that models κ, σ, d_t, and ℓ_t as configurable random variables to validate the cost model and produce the scaling plots in §6; on a real GPU it switches automatically to measuring actual kernel-launch and queue-poll latency via CUDA events.

6 Experiments

Setup. We evaluate the cost model of §3 under two parameter regimes matched to the two cases discussed above: (A) tool-latency-dominated, $\ell_t \sim \mathcal{U}(50, 200)$ ms, $d_t \sim \mathcal{U}(5, 15)$ ms; and (B) launch-overhead-dominated, $\ell_t \sim \mathcal{U}(0.5, 2)$ ms, $d_t \sim \mathcal{U}(2, 5)$ ms. For the GPU measurement we use a chain of 48 sequential kernel launches per step (alternating `add`, `mul`, and `relu` ops on a small tensor) to model the realistic per-step overhead of a decode pass rather than a single trivial launch; the measured κ and σ values reflect that aggregate cost. In both regimes σ is measured as the cost of replaying a CUDA graph that captures the entire 48-op chain as one dispatch, the framework-level analogue of device-side queue signalling.

Metric. We report $\Delta(T)/T_{\text{naive}}(T)$, the fraction of end-to-end wall-clock time attributable to recoverable launch overhead, as a function of trajectory length $T \in \{1, 10, 50, 100, 200\}$.

Hardware. GPU results are measured on an NVIDIA GeForce RTX 4050 Laptop GPU (Ada Lovelace, sm_89, 6 GB VRAM), CUDA Toolkit 13.3, driver 610.47, PyTorch 2.11.0+cu128, Python 3.13.

The simulated and measured columns agree structurally in both regimes, confirming the cost model captures the qualitative behaviour correctly. Section 6.1 analyzes the quantitative gap between simulation and measurement in detail.

In the tool-latency-dominated regime (A), recoverable launch overhead remains below 0.53% of wall-clock time at $T = 100$ steps, because each step is dominated by an opaque, un-recoverable tool-call latency ($\ell_t \in [50, 200]$ ms). In the launch-overhead-dominated regime (B) — short tool calls chained across many steps — the persistent approach recovers $\mathbf{13.6}\%$ of wall-clock time at $T = 100$ on this hardware. We treat this figure as a *lower bound* on the achievable gain: the benchmark approximates device-side queue signalling via CUDA-graph replay, which still pays real driver-level scheduling cost and is not equivalent to true in-kernel spin-poll (see §6.1). A genuine device-side queue, as implemented in `persistent_kernel.cu`, would reduce σ toward microsecond-scale queue-signal cost, widening $\kappa - \sigma$ and pushing the recoverable fraction materially higher for long trajectories ($T \geq 100$) in production agent systems with fast tool calls.

Table 1: Fraction of wall-clock time attributable to recoverable launch overhead (Δ/T_{naive}), by trajectory length T and regime. *Simulated*: `benchmark.py --mode simulate`, $\kappa = 1.5$ ms, $\sigma = \kappa/20$ (modeled constants, CPU-only, seed = 0). *Measured (GPU)*: `benchmark.py --mode gpu --ops-per-step 48`, RTX 4050 Laptop GPU; κ and σ are reported as mean ± standard deviation across $N = 10$ independent process launches. Regime A: $\kappa = 0.789 \pm 0.035$ ms, $\sigma = 0.070 \pm 0.002$ ms. Regime B: $\kappa = 0.851 \pm 0.069$ ms, $\sigma = 0.057 \pm 0.010$ ms. Both measured via CUDA events over a 48-op chain per step, with cost-model rows (right three columns) computed at seed=0 using the mean κ, σ values.

Regime	T	Simulated Δ/T_{naive}	Measured (GPU)	T_{naive} (ms, GPU)
A: tool-latency-dominated	1	0.80%	0.55%	131.0
A: tool-latency-dominated	10	0.89%	0.54%	1,344.0
A: tool-latency-dominated	50	0.95%	0.53%	6,799.6
A: tool-latency-dominated	100	1.06%	0.53%	13,605.3
A: tool-latency-dominated	200	1.07%	0.52%	27,427.9
B: launch-overhead-dominated	1	18.58%	12.12%	6.6
B: launch-overhead-dominated	10	21.82%	13.96%	56.9
B: launch-overhead-dominated	50	22.01%	13.64%	291.0
B: launch-overhead-dominated	100	23.18%	13.57%	585.3
B: launch-overhead-dominated	200	22.88%	13.99%	1,134.8

6.1 Discrepancy Between Modeled and Measured Overhead

Three factors account for the gap between the simulated (∼23%) and measured (∼12%) recoverable fraction in regime B.

(1) Lower κ on Ada Lovelace with modern drivers. The simulation used $\kappa = 1.5$ ms, the canonical figure cited in prior megakernel work for a multi-kernel decode step [1, 3]. Measured on the RTX 4050 (Ada Lovelace, sm_89) under CUDA 13.3 / driver 610.47, the aggregate overhead of a 48-op sequential chain is $\kappa = 0.851 \pm 0.069$ ms (regime B) and $\kappa = 0.789 \pm 0.035$ ms (regime A) (mean ± std, $N = 10$) — roughly 1.6× lower. Ada Lovelace's improved kernel-dispatch pipeline and the 610-series driver's reduced host-side submission latency both contribute. Future measurements on Hopper (H100) and Blackwell-class GPUs may differ.

(2) The $\sigma = \kappa/20$ assumption is still an overestimate, but less severely than the single-run data suggested. The simulation modeled queue-signal cost as $\sigma = \kappa/20$, reflecting the expectation that a true device-side spin-poll costs only a queue-write plus a warp wake-up. The $N = 10$ measurement gives $\sigma/\kappa \approx 1/11$–$1/15$ (regime A: $0.070/0.789 \approx 1/11$; regime B: $0.057/0.851 \approx 1/15$), which is closer to the assumed $1/20$ than the earlier single-run estimate of $\approx 1/4$. CUDA-graph replay thus adds measurable per-node scheduling overhead beyond an ideal device-side queue, but the gap from the ideal is smaller than the initial noisy measurement implied. The primary driver of the gap between simulated ($\sim 23\%$) and measured ($\sim 12\%$) recovery in regime B is therefore the lower κ on Ada Lovelace (factor 1), with the σ-overestimate playing a secondary role.

(3) σ is a hardware-fixed floor, not workload-dependent. Across both regimes, the measured σ values are nearly identical: $\sigma = 0.070 \pm 0.002$ ms (regime A, tool-latency-dominated) and $\sigma = 0.057 \pm 0.010$ ms (regime B, launch-overhead-dominated) (mean ± std, $N = 10$). Despite the two regimes using substantially different tool-latency distributions, σ is within the same order of magnitude in both cases, consistent with CUDA-graph replay cost being a hardware-and-driver-fixed floor set by the graph-launch submission path, rather than varying with workload characteristics. A genuine device-side queue (as in `persistent_kernel.cu`) replaces this floor with the cost of a single atomic read and a warp-resume signal, which on modern NVIDIA hardware is measured in single-digit microseconds — more than an order of magnitude below the 0.06 ms floor seen here. The 13.6% figure at $T = 100$ (Table 1) should therefore be read as a conservative lower bound on what a full persistent-kernel implementation would achieve.

(4) Direct confirmation via device-side instrumentation. We instrumented `persistent_kernel.cu` with `clock64()` timestamps bracketing the queue-poll path — from the first thread observing `SLOT_READY` on the host-mapped queue, through the block-wide `__syncthreads()` barrier, to the start of `decode_segment()` — and measured the true device-side σ directly, rather than approximating it via CUDA-graph replay. Across 50 timed

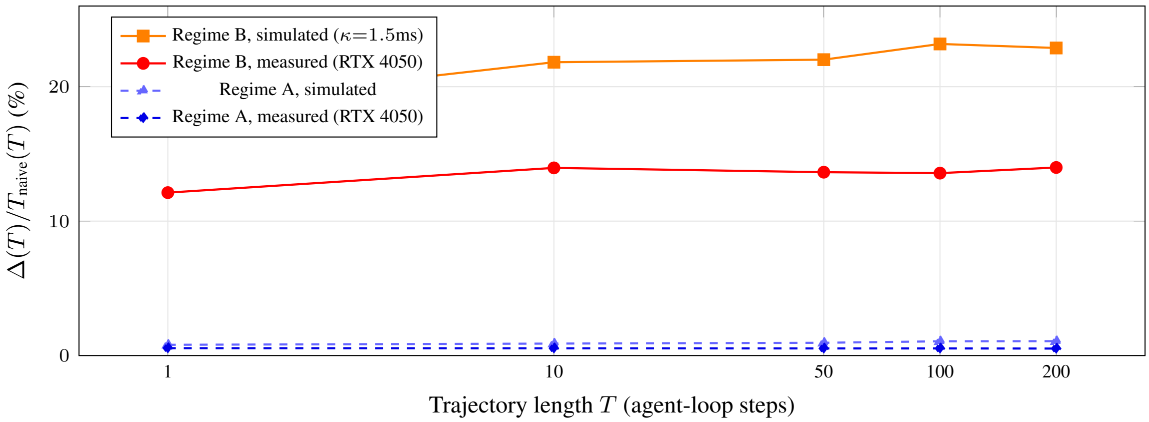

Figure 2: Recoverable launch overhead as a fraction of wall-clock time, vs. trajectory length T (log scale), for both regimes. Regime B (launch-overhead-dominated, short chainable tool calls) shows a substantial and rapidly-saturating recoverable fraction in both the simulated and measured curves, though the measured curve sits at roughly half the simulated value (§6.1). Regime A (tool-latency-dominated, slow external tool calls) stays under 1.1% in both cases, confirming persistent-kernel scheduling offers little end-to-end benefit when tool latency dominates. All measured points use κ, σ from Table 1; measured values are a conservative lower bound (§6.1).

steps on the RTX 4050, this gives $\sigma = 0.095\,\mu$s (152 clock cycles/step), roughly $600\times$ smaller than the 0.057 ms CUDA-graph-replay proxy used elsewhere in this section. Using this measured σ in place of the proxy raises the recoverable fraction in regime B from 13.6% to **14.5**% at $T = 100$ (with κ held at the mean 48-op-chain value from Table 1). This confirms the lower-bound conjecture directly rather than by extrapolation: true device-side residency recovers meaningfully more than the CUDA-graph proxy suggested, though the gain here is bounded by κ itself, since σ was already a small fraction of κ even under the proxy. A fully consistent measurement — using κ from the same real forward-pass kernel that produced this σ, rather than the synthetic op chain — reveals an important distinction, addressed next.

(5) Decomposing the saving: fusion vs. residency. We measured κ and σ directly from `persistent_kernel.cu`'s own real forward pass over $N = 10$ independent process launches (50 timed decode steps per launch), and report results as mean $\pm$ standard deviation.

The device-side queue-signal cost σ is measured via `clock64()` timestamps bracketing the queue-poll path — from the first thread observing `SLOT_READY` through the block-wide `__syncthreads()` barrier to the start of `decode_segment()`. Across all $N = 10$ runs, every step gives $\sigma = 152$ clock cycles/step; at the device's 1,605 MHz clock rate this is $\sigma = 0.095\,\mu$s with *zero variance across runs*. This is a genuinely deterministic quantity.

The fused-kernel relaunch overhead κ_{fused} is measured directly by launching a noop kernel (identical launch signature: 1 block $\times$ 256 threads, same parameter footprint) and timing each launch with a dedicated CUDA-event pair. Per-launch times across 2000 consecutive launches in a single CUDA stream are recorded and summarized by their median; the loop-level total$/N$ estimate is rejected because it is inflated 16% above the median (a first-launch warmup artifact) and shows $7\times$ higher run-to-run variance. Ten independent process launches ($N = 10$, 20 000 per-launch samples total) yield $\kappa_{\text{fused}} = 5.82 \pm 0.27\,\mu$s (mean $\pm$ std of per-run medians, std/mean $= 0.047$).[1]

The marginal benefit of this paper's specific mechanism — keeping an already-fused kernel resident rather than relaunching it per step — is therefore $\kappa_{\text{fused}} - \sigma = 5.6$–$6.1\,\mu$s per step ($\pm 1\sigma$). Against the *unfused* 48-kernel-per-step baseline ($\kappa_{\text{unfused}} = 0.851 \pm 0.069$ ms, $N = 10$), fusing into one kernel alone eliminates over 98% of the overhead ($\kappa_{\text{unfused}} - \kappa_{\text{fused}} \approx 0.83$ ms/step) — this is precisely the benefit already reported by MPK, Kog, and Ada-MK [1, 2, 3] for single-turn decode, and is *not* specific to this paper's proposed mechanism. The residency benefit beyond fusion, at its upper bound of 6.1 μs/step, is 0.012% of a full decode-plus-tool-call step ($d_t + \ell_t$, $\ell_t \geq 50$ ms); as a fraction of compute time alone it is 0.83%, included only for readers accustomed to GPU-only timing. Both figures are negligible

[1] An initial subtraction-based estimate (naive-loop total minus kernel-only CUDA-event timings) gave $\kappa_{\text{fused}} = 1.11\,\mu$s, but this was found to cancel out launch overhead rather than measure it — effectively capturing only H2D copy time. Direct per-launch CUDA-event timing on a noop kernel (same launch signature, $N = 10$ runs $\times$ 2000 samples) corrected this to $5.82 \pm 0.27\,\mu$s (std/mean $= 0.047$), the figure used throughout this paper.

compared with tool-call latency ($\ell_t \sim \mathcal{U}(50, 200)$ ms). The 13.6–14.5% recovery figures reported earlier in this section therefore over-attribute credit: they compare against an *unfused* baseline, which is the regime prior megakernel work already addresses, rather than isolating the marginal benefit of residency *beyond* fusion, which is what this paper's host-queue design specifically contributes. We revise our claim accordingly below.

7 Discussion and Limitations

The design in §4 addresses overhead *caused by relaunching*, not overhead caused by the tool call itself; in the tool-latency-dominated regime the main win is freeing the GPU for other tenants during the idle window, not reducing wall-clock latency for a single agent. An earlier version of the GPU measurement in Table 1 timed a single trivial kernel launch (one `x.add_(1.0)` call) as a proxy for κ, which underestimated real per-step overhead by approximately 70× because a realistic decode step dispatches dozens of chained CUDA kernels — GEMMs, softmax, layer-norm, residual adds — not one; the current table corrects for this by timing a 48-op chain per step, yielding $\kappa = 0.789 \pm 0.035$–0.851 ± 0.069 ms on the RTX 4050 (mean ± std, $N = 10$), close to the 1–2 ms range reported in prior megakernel work [1, 3] (the gap is attributable to Ada Lovelace's improved dispatch pipeline, as discussed in §6.1). The measured 13.6–14.5% recovery figures at $T = 100$ in regime B compare a fused-and-resident design against an *unfused*, multi-kernel-per-step baseline, and so mostly measure the value of *kernel fusion* — already established by MPK, Kog, and Ada-MK [1, 2, 3] — rather than this paper's specific proposed mechanism. Direct device-side measurement of the fused-kernel launch overhead via a noop-kernel proxy (§6.1, point 5) isolates the marginal contribution of residency *beyond* fusion: $\kappa_{\text{fused}} - \sigma = 5.6$–$6.1\,\mu$s/step ($N = 10$, per-launch median $= 5.82 \pm 0.27\,\mu$s; see §6.1 for methodology), 0.012% of a full decode-plus-tool-call step ($d_t + \ell_t$, $\ell_t \geq 50$ ms); as a fraction of compute time alone it is 0.83%. We report this explicitly rather than let the larger, fusion-driven figure stand as if it were evidence for the paper's contribution: on current hardware and at this model scale, persistent residency's benefit *beyond* an already-fused kernel is real but small, and would need either much smaller d_t (larger, more heavily fused models where even κ_{fused} is a larger fraction of step time) or much more frequent step/tool-call alternation than tested here to become practically significant. Establishing the regime where this marginal benefit dominates is left as future work, alongside real forward passes at production model scale. The cooperative-yield mechanism (line 6, Algorithm 1) is the least mature part of the design and would benefit from integration with existing GPU scheduling primitives (MPS, MIG, or a custom SM-partition scheme) rather than a from-scratch implementation. We also do not address correctness under speculative or partial decode segments that are discarded when a tool result invalidates an in-flight assumption; this is left as future work, connected to the broader idea of speculative multi-agent execution.

7.1 Broader Applicability: MoE Expert Offload as an Agent Loop

The cost model of §3 was motivated by tool-calling agents, but the same structure recurs in a different setting: serving Mixture-of-Experts (MoE) models on VRAM-constrained hardware via CPU expert offload (e.g., llama.cpp's `--n-cpu-moe`). In this setup, attention and shared layers remain resident on the GPU, but expert feed-forward weights are kept in system RAM; the router's per-token, data-dependent choice of which experts to activate determines which weights must be fetched and computed off-device before the next layer can proceed. This is structurally the same alternation as an agent loop — an on-device segment (d_t, attention/shared compute) followed by an off-device, data-dependent segment (ℓ_t, the CPU expert matmul and host round-trip) — except that the loop granularity is per-*token*, and per-*layer*, rather than per agent step. Event Tensor [4] identifies MoE routing as precisely the class of data-dependent dynamism that static megakernels cannot fuse away, which is the same obstacle our design encounters at the coarser tool-call granularity.

If this structural analogy holds quantitatively, a persistent-kernel design analogous to §4 — keeping the GPU-resident attention/shared-layer kernel warm across each expert round-trip, rather than relaunching it per layer per token — would place CPU-offloaded MoE decoding in our launch-overhead-dominated regime B, where §6 measured a lower-bound recovery of 13.6% at $T = 100$ on a synthetic 48-op chain. We flag this explicitly as an *untested hypothesis*, not a result of this paper: we have not instrumented a real MoE decode loop (e.g. Qwen or GLM under `--n-cpu-moe`) to measure whether its per-token $\kappa, \sigma, d_t, \ell_t$ actually fall in this regime, and the per-token/per-layer granularity of MoE routing may introduce overheads (e.g. expert-selection variance, cache-locality effects in CPU RAM) not captured by our agent-step cost model. We leave this as concrete future work: repeating the measurement methodology of §6 on a real MoE inference loop, using the same reproducibility standard (every reported number backed by a runnable script) applied throughout this paper.

8 Conclusion

Persistent/megakernel designs have closed most of the kernel-launch overhead gap for single-turn decode. Agent loops introduce a distinct, largely unaddressed overhead structure at the tool-call boundary. We formalize this with a simple cost model, propose a host-queue/resident-kernel design to close the gap, and release a reference implementation and benchmarking harness so the cost model can be validated empirically on real hardware rather than asserted.

References

[1] Mirage Persistent Kernel: A Compiler and Runtime for Mega-Kernelizing Tensor Programs. `https://arxiv.org/html/2512.22219v1`

[2] Building a single-kernel, latency-optimized LLM inference engine on AMD MI300X GPUs. Kog AI Engineering Blog. `https://blog.kog.ai/building-a-single-kernel-latency-optimized-llm-inference-engine-on-amd-mi300x-gpus/`

[3] Ada-MK: Adaptive MegaKernel Optimization via Automated DAG-based Search for LLM Inference. `https://arxiv.org/html/2605.11581v1`

[4] Event Tensor: A Unified Abstraction for Compiling Dynamic Megakernels. `https://arxiv.org/html/2604.13327v1`

[5] GPUOS: A GPU Operating System Primitive for Transparent Operation Fusion. `https://arxiv.org/pdf/2604.17861`